

CONESTA FORGE

Build Your Own AI Web Application

Trainer's Lesson Plan & Student Guide

A 5-day, hands-on course · 10 lessons

June 2026

Stack: Next.js · MongoDB Atlas (Mongoose) · Vercel · Groq | no auth (single-user build)

Contents

Course Overview

What learners can do by the end

How This Course Works

How to use this guide

The five-day arc

Day 1 — Spec & Foundation

Lesson 1.1 — Setup & Your One-Page Spec

Lesson 1.2 — Scaffolding a Web App with AI

Day 2 — Build the Core App

Lesson 2.1 — Data & CRUD with MongoDB

Lesson 2.2 — Deploy to Vercel

Day 3 — Add the Brain

Lesson 3.1 — Getting an LLM Into Your App

Lesson 3.2 — Prompts & Key Safety

Day 4 — Build Your AI Feature

Lesson 4.1 — Designing Your Signature Feature

Lesson 4.2 — Wiring the Feature End-to-End

Day 5 — Polish & Ship

Lesson 5.1 — Guardrails & Reliability

Lesson 5.2 — Ship & Share

Glossary

Stack Cheat-Sheet

Course Overview

Build Your Own AI Web Application is a five-day, beginner-friendly course in which each learner ships one small, real web app that ends with a working AI feature. It is hands-on: learners direct an AI coding assistant to write most of the code, while the course teaches the judgment — what to build, how to structure it, how to call an LLM safely, and how to ship.

Audience. Beginners who can follow instructions and are comfortable installing tools. No prior web or AI experience assumed.

The stack (one fixed, known-good path).

Layer	Tool	Role
Framework	Next.js (React + TypeScript, App Router)	UI pages + backend API routes in one project
Database	MongoDB Atlas via Mongoose	Cloud document database for the app's data
Deploy	Vercel	Auto-deploys the GitHub repo to a public URL
AI	Groq (groq-sdk)	Fast LLM API with a generous free tier

Supporting tools: Git + GitHub (version control + the repo the judge reads), Node.js (runs the app), and an AI coding assistant (Cursor, GitHub Copilot, or Claude/ChatGPT). **This build has no login** — it is intentionally single-user so beginners can ship in five days.

Shape. 5 days x 2 lessons = 10 lessons. Each day has a theme and a concrete deliverable. A learner finishes the week with a deployed, working AI app, a README, a demo clip, and a share post.

What learners can do by the end

- Scope a small product and freeze a one-page spec.
- Build a data model and CRUD API on MongoDB with Next.js.
- Deploy to a public URL and debug the common deploy failures.
- Call an LLM safely from the server and design a prompt for a real job.
- Build, guard, and ship an AI feature that runs on real user data.

How This Course Works

Each lesson is a **card deck**, in this order:

1. **Intro** — the hook: why this lesson matters in one line.
2. **Chapters** — the teaching content, broken into short sections. Most chapters end with a **knowledge check** (a multiple-choice question the learner must answer to move on).
3. **Video** — a short (~2.5-3 min) recap video at the **end** of the lesson. It is a **required gate**: the learner must watch it through (no fast-forwarding) before the lesson can be completed.
4. **Recap / Complete** — key points and the move to the next lesson.

Completion is gated, server-side. A lesson cannot be marked complete (and the next one cannot open) until the required video has actually been watched to roughly 95%. Skipping ahead is blocked; rewinding to rewatch is allowed.

Days unlock on the calendar. For an enrolled event, a day's lessons unlock on that event day. Future days are locked ("Unlocks on Day N"); the current day and all past days stay open so learners can catch up. A typical day is ~35-45 minutes of reading plus two short videos.

How to use this guide

- **Trainers:** each lesson below lists what is taught (chapter by chapter), what to emphasize, the common pitfalls, a rough time plan, a discussion prompt, and the knowledge-check questions with answers so you can quiz the room.
- **Students:** read the day goal and the "what you will have built" line to know where you are headed, then the lesson walkthroughs to see what each lesson covers.

The five-day arc

Day	Theme	You ship	Rubric
1	Spec & Foundation	A one-page spec	README states the spec + screens; a committed, runnable app skeleton.
2	Build the Core App	One mongodb entity with crud through next.js api routes	A working CRUD entity; deploy configuration committed; a reachable live URL.
3	Add the Brain	A groq call wired into a next.js api route	groq-sdk imported; key read from an env var; a real call site that returns model text.
4	Build Your AI Feature	The one ai feature	The AI feature is implemented end to end and does the job its name promises.
5	Polish & Ship	Guardrails on the ai call	Error handling/guardrails on the AI call; a working deployed demo; a clear README; a share post.

Day 1 — Spec & Foundation

Goal. Decide exactly what you are building, then stand up a skeleton that runs.

What the learner ships today. A one-page spec, a scaffolded Next.js app that boots, a GitHub repo, and a README that states the spec.

Day rubric (how it is judged). README states the spec + screens; a committed, runnable app skeleton.

By end of day. The student leaves with a frozen project idea and an app that runs, in a repo with a README.

Lesson 1.1 — Setup & Your One-Page Spec

~24 min reading · ends with a required ~2:58 video

Hook. Nobody fails this event because they can't code. They fail because they burned a day fighting an install, or never decided what they were building. Get setup and a one-page spec done now and the whole week opens up.

In one line. Install Git + Node, pick an AI coding assistant, and create four free accounts — GitHub, MongoDB Atlas, Vercel, and Groq. Then write a one-page spec that freezes what you are building. End state: tools ready, accounts ready, an empty repo, and a decided plan.

Learning objectives. - **One default stack** — Next.js + MongoDB Atlas (Mongoose) + Vercel + Groq. Follow it unless you already know another well. - **Install + four accounts** — Git + Node installed; GitHub, MongoDB Atlas, Vercel, Groq accounts; one empty GitHub repo. - **Freeze a one-page spec** — Data, fields, screens, AI job — decided once. If it spills past one page, you are over-scoping.

Walkthrough (what is taught). 1. **The default stack and your tools** — This course gives you ONE concrete stack so you never have to stop and choose. 2. **Accounts and an empty repo** — You need four free accounts. 3. **The frozen contract** — For the next five days you build a single, small, real web application — and by Day 4 it does something genuinely useful with an AI model. 4. **Writing your one-page spec** — Open a blank document and write five short sections.

Required video. A ~2:58 recap plays at the end; the learner must watch it through to complete the lesson and continue.

Teaching notes. - *Emphasize:* The spec is the highest-leverage 15 minutes of the week. Do not let learners jump to code. Make each one write the sentence: "the model takes ____ and returns ."
- **Common pitfall: Over-scoping. If a spec spills past one page it is too big. Watch for "and also a chat, and a calendar, and..."**. - **Time plan:** Treat installs + accounts as pre-work if you can (~15 min). Spend live time on the frozen contract and the one-page spec (~25 min). - **Discussion prompt:** Go around the room: each student says their app in one sentence, "X lets you ____ so that ."

Knowledge checks (with answers). 1. *What is the course's default stack you should use unless you already know another?* - **Answer:** Next.js, MongoDB Atlas (with Mongoose), Vercel, and Groq - **Why:** Next.js (UI + API), MongoDB Atlas (data via Mongoose), Vercel (deploy), Groq (AI) is the known-good default. Swap only if you already know an alternative. 2. *Which four free accounts should you create during setup?* - **Answer:** GitHub, MongoDB Atlas, Vercel, and Groq - **Why:** GitHub (repo), MongoDB Atlas (database), Vercel (deploy), and Groq (AI) are the four accounts. Node/Git/editors are installed tools, not accounts. 3. *What does it mean to "freeze the contract"?* - **Answer:** Make the core decisions once and stop renegotiating them mid-build - **Why:** Freezing means decided — not perfect, and not "no code yet." It stops Day-3 changes from rippling through everything. 4. *Which belongs in your one-page spec?* - **Answer:** The one or two data entities and their fields - **Why:** Title, pitch, user, data, and the AI feature — one page. Styling is Day 5; model settings come later; secrets never go in a spec.

Resources. - Next.js — official docs: <https://nextjs.org/docs> - Node.js downloads (LTS): <https://nodejs.org/en/download> - MongoDB Atlas — get started: <https://www.mongodb.com/atlas/database> - Groq Console: <https://console.groq.com>

Lesson 1.2 — Scaffolding a Web App with AI

~20 min reading · ends with a required ~2:23 video

Hook. An AI builder is a power tool. Point it precisely and it scaffolds in minutes; wave it vaguely and you get a generic mess you have to throw away.

In one line. Scaffold a Next.js app with one command, then direct your AI assistant one concrete screen at a time against your frozen contract. End Day 1 with a running skeleton committed to your GitHub repo and a README that states the spec.

Learning objectives. - **Scaffold with one command** — `npx create-next-app` gives a runnable Next.js app — UI and API routes in one project. - **Concrete, additive prompts** — Name the file, the data source, the data shape, the boundary — one screen at a time. - **Running > pretty** — End Day 1 with a booting skeleton + repo + README stating the spec. Polish is Day 5.

Walkthrough (what is taught). 1. **Scaffolding the Next.js project** — You won't build the project structure by hand — one command generates a complete, runnable Next.js app. 2. **Prompting your AI assistant effectively** — Your AI assistant turns plain-English instructions into code — but quality is a direct function of how concretely you ask. 3. **Your first runnable skeleton** — By end of Day 1 you want a skeleton: the app boots, you can navigate between your core screens, and at least one screen shows real (even if hard-coded) data. 4. **Committing to your repo with a README** — Everything lives in the GitHub repo you created earlier — and on Day 5 the AI judge reads it.

Required video. A ~2:23 recap plays at the end; the learner must watch it through to complete the lesson and continue.

Teaching notes. - *Emphasize:* Concrete prompting. Show the weak vs strong prompt side by side: name the file, the data source, the data shape, and the boundary. - *Common pitfall:* Chasing polish on Day 1. A grey skeleton that runs beats a pretty screen that will not boot. - *Time plan:* ~5 min scaffold, ~10 min prompting demo, ~5 min first commit + README. - *Discussion prompt:* Everyone shares their repo URL and the first paragraph of their README.

Knowledge checks (with answers). 1. *What does `npx create-next-app` give you, and why does it suit a beginner?* - **Answer:** A runnable project with both UI pages and backend API routes in one place - **Why:** One command scaffolds a runnable Next.js app where pages and server-side API routes live together — no separate backend to set up. 2. *Which is the stronger prompt to your AI assistant?* - **Answer:** "Create a page at `app/habits/page.tsx` that fetches GET `/api/habits` and renders each as a card with name and streak" - **Why:** Naming the file, data source, and shape — one screen at a time — gives the assistant what it needs. Vague or "do everything" prompts produce generic code. 3. *What is the right Day-1 milestone for your skeleton?* - **Answer:** It boots and you can navigate core screens, with at least one showing real data - **Why:** A booting, navigable skeleton with one real-data screen is the goal. Styling is Day 5; the AI call is Day 3-4. 4. *To pass today's rubric, what must your repo contain?* - **Answer:** A scaffolded, runnable app skeleton plus a README stating the spec and screens - **Why:** The Day-1 judge wants a README that states the spec and screens, and a committed, runnable app skeleton. Not a finished app, but more than just a README.

Resources. - create-next-app reference: <https://nextjs.org/docs/app/api-reference/cli/create-next-app> - Next.js Project Structure: <https://nextjs.org/docs/app/getting-started/project-structure> - GitHub — create a repo: <https://docs.github.com/en/get-started/quickstart/create-a-repo>

Day 2 — Build the Core App

Goal. Make the app store real data and put it on the public internet.

What the learner ships today. One MongoDB entity with CRUD through Next.js API routes, deployed live on Vercel.

Day rubric (how it is judged). A working CRUD entity; deploy configuration committed; a reachable live URL.

By end of day. The student has a deployed app that creates and lists real data persisted in MongoDB Atlas.

Lesson 2.1 — Data & CRUD with MongoDB

~22 min reading · ends with a required ~3:13 video

Hook. Almost every web app is the same four verbs on some kind of "thing": create it, read it, update it, delete it. Master that on MongoDB and you can build most of what exists.

In one line. Model your one core entity as a Mongoose schema, connect to a free MongoDB Atlas cluster, then implement Create, Read, Update, and Delete through Next.js API routes — with data persisting in the cloud and flowing cleanly from server to screen.

Learning objectives. - **One clean Mongoose model** — Schema with the few fields your screens need; guard with `mongoose.models.Habit || mongoose.model(...)`. - **Connect to Atlas, cached** — Read `MONGODB_URI` server-side, cache the connection for serverless, and never commit the string. - **CRUD = route handlers** — GET/POST in `app/api/habits/route.ts`; Create + Read first. Data persists in the cloud.

Walkthrough (what is taught). 1. **Modelling your one core entity** — Your app stores some kind of "thing" — a habit, a post, a recipe, a logged expense. 2. **Connecting to MongoDB Atlas** — Your database lives in the cloud on **MongoDB Atlas**, and you reach it with one secret string. 3. **The four verbs: Create, Read, Update, Delete** — "CRUD" is the four things you do to your entity, covering almost everything a basic app needs. 4. **Wiring the screen, and proving persistence** — Your pages talk to your API routes over HTTP.

Required video. A ~3:13 recap plays at the end; the learner must watch it through to complete the lesson and continue.

Teaching notes. - *Emphasize:* Two beginner traps live here: the model-guard line (`mongoose.models.Habit || mongoose.model(...)`) for hot reload, and the cached connection for serverless. Call out both. - *Common pitfall:* Modeling five entities (over-scope) and forgetting "await `dbConnect()`" at the top of each route. - *Time plan:* ~8 min model + connection, ~8 min CRUD routes, ~6 min wire the screen + the persistence test (restart and reload). - *Discussion prompt:* What is your one entity and its two or three fields?

Knowledge checks (with answers). 1. *How should you model the data for a typical five-day app?* - **Answer:** One clean entity (sometimes two that relate), as a Mongoose schema with a few fields - **Why:** One well-modelled Mongoose entity carries a real app and its AI feature. Many collections is over-scope; an in-memory variable loses data on restart. 2. *Why does the `MONGODB_URI` go in `.env.local` rather than in your code?* - **Answer:** It's a secret — it contains your database password, and committed code gets read by others and the judge - **Why:** The URI carries your DB user and password; a hard-coded URI is a leaked secret. It lives in `.env.local` (gitignored) and is read with `process.env` server-side. 3. *Where do your CRUD verbs live in a Next.js app, and which do you build first?* - **Answer:** As handlers in `app/api/.../route.ts` files that call the Mongoose model; Create and Read first - **Why:** GET/POST/PATCH/DELETE handlers live in `app/api/.../route.ts` and call `Habit.find/create/etc`. Build Create + Read first — you can't test update/delete without data. 4. *What actually proves your data persists?* - **Answer:** Create an item, restart `npm run dev`, reload — it's still there (it lives in Atlas) - **Why:** Surviving a server restart means the document lives in MongoDB Atlas — durable and real. On-screen-once or a cleared form prove nothing about persistence.

Resources. - MongoDB Atlas — create a cluster: <https://www.mongodb.com/docs/atlas/getting-started/> - Mongoose — schemas & models: <https://mongoosejs.com/docs/guide.html> - Next.js Route Handlers: <https://nextjs.org/docs/app/building-your-application/routing/route-handlers>

Lesson 2.2 — Deploy to Vercel

~18 min reading · ends with a required ~2:27 video

Hook. An app on your laptop is a science project. An app with a public URL is something a stranger can actually use — and a judge can actually open.

In one line. Deploy to Vercel today, while it is low-stakes: push your code, import the repo, and beat the three walls every beginner hits — env vars, MongoDB Atlas network access, and the connection string — so "works on my machine" never becomes a Day-5 surprise.

Learning objectives. - **Deploy on Day 2** — Required today, and the first deploy is fiddly — fight it while it's low-stakes, not at the deadline. - **Re-add env vars in Vercel** — `.env.local` isn't

uploaded; set MONGODB_URI (and later GROQ_API_KEY) in Vercel settings. - **Open Atlas to Vercel** — Add 0.0.0.0/0 in Atlas Network Access, or the deployed app's DB calls hang.

Walkthrough (what is taught). 1. **Why deploy on Day 2** — Deploying puts your app on a public URL anyone can open, and it is **a firm requirement, not "ideally."** Today's rubric wants deploy configuration committed and the app reachable — so you deploy on Day 2, while it... 2.

Deploying to Vercel — To do: push your code to GitHub, go to vercel.com, click **Add New Project**, import your repo, and deploy. 3. **The three deploy walls** — Three things commonly break on the first deploy. 4. **Test like a stranger** — A deploy is not done when the build is green; it is done when it works for someone who is not you.

Required video. A ~2:27 recap plays at the end; the learner must watch it through to complete the lesson and continue.

Teaching notes. - *Emphasize:* Deploy early while it is cheap. Drill the three walls: env vars are not uploaded, Atlas Network Access must allow 0.0.0.0/0, and the connection string must be exact. - *Common pitfall:* "Works locally, fails deployed" — almost always an env var or Atlas network access. Always test in a fresh/incognito window. - *Time plan:* ~5 min deploy, ~8 min the three walls, ~5 min test-like-a-stranger. - *Discussion prompt:* Pair up and open each other's live URLs; add an item and reload.

Knowledge checks (with answers). 1. *Why deploy on Day 2 instead of waiting until Day 5?* - **Answer:** Deploy config is required today, and the first deploy is fiddly — fight it while it's low-stakes - **Why:** The Day-2 rubric requires deploy config present, and the first deploy (env vars, Atlas access) is fiddly — solve it early, not at the deadline. 2. *Once your GitHub repo is connected to Vercel, what happens when you push to main?* - **Answer:** Vercel automatically rebuilds and redeploys the live site - **Why:** A connected repo means every push to main triggers a fresh build and deploy — write, commit, push, and the live site updates itself. 3. *Your deployed app times out connecting to MongoDB, but it works on your laptop. Most likely cause?* - **Answer:** Atlas Network Access doesn't allow Vercel's servers — add 0.0.0.0/0 (and confirm the env var is set) - **Why:** Atlas blocks unknown IPs by default; your home IP may be allowed but Vercel's isn't. Add 0.0.0.0/0 in Network Access (and make sure MONGODB_URI is set in Vercel). 4. *How should you test that your deploy actually works?* - **Answer:** Open the public URL in a fresh/incognito browser and run the core loop (add an item, reload, confirm it stuck) - **Why:** Only the live URL, exercised as a stranger, counts. A green build can still fail at runtime (env vars, Atlas access); exercise the real loop in a fresh browser.

Resources. - Deploy Next.js on Vercel: <https://nextjs.org/docs/app/building-your-application/deploying> - Vercel — Environment Variables: <https://vercel.com/docs/projects/environment-variables> - Atlas — configure Network Access: <https://www.mongodb.com/docs/atlas/security/ip-access-list/>

Day 3 — Add the Brain

Goal. Make the first real LLM call from the app, safely, server-side.

What the learner ships today. A Groq call wired into a Next.js API route, with the key read from an environment variable.

Day rubric (how it is judged). groq-sdk imported; key read from an env var; a real call site that returns model text.

By end of day. The student's app can send text to an LLM and get text back, with the key kept secret.

Lesson 3.1 — Getting an LLM Into Your App

~19 min reading · ends with a required ~3:01 video

Hook. A large language model is a function you call over the internet: text goes in, text comes out. Today you make your first call — and your app stops being ordinary.

In one line. An LLM API is a service you send text to and get text back from. Get a Groq API key, install the groq-sdk, and make one real call from a Next.js API route — the foundation of every AI feature.

Learning objectives. - **Text in, text out** — An LLM API takes a prompt and returns a completion. No neural-net knowledge needed. - **The key is a secret** — Generate a Groq gsk- key; treat it like a password. Never in code or your repo. - **groq-sdk, server-side** — npm install groq-sdk; call it from a Next.js API route — one controlled place the AI lives.

Walkthrough (what is taught). 1. **What an LLM API actually is** — A large language model (LLM) is the engine behind tools like ChatGPT. 2. **Getting your Groq API key** — To call a model you need an **API key** — a secret string that authorises your requests and tracks usage. 3.

Installing the groq-sdk — You *could* call the API with raw HTTP, but Groq publishes an **SDK** — a small library that wraps the API in convenient functions — and it is far easier. 4. **Server-side, and your first real call** — Keep the AI call on the **server side** — in a Next.js API route, never in the browser.

Required video. A ~3:01 recap plays at the end; the learner must watch it through to complete the lesson and continue.

Teaching notes. - *Emphasize:* Demystify: an LLM API is text in, text out over the internet. The key is a secret from the moment it is copied. - *Common pitfall:* Putting the call (or the key) in the browser. The call must be server-side, in an API route. - *Time plan:* ~5 min concept, ~5 min key + SDK install, ~9 min the first real call. - *Discussion prompt:* What will your one prompt take in, and what should it return?

Knowledge checks (with answers). 1. *What is the simplest accurate model of an LLM API?* - **Answer:** You send text (a prompt) and get text (a completion) back over the internet - **Why:** Text in, text out over the internet. You don't run the model yourself or need neural-net theory — it's a hosted service you call. 2. *How should you treat your Groq API key once you copy it?* - **Answer:** Like a password — kept secret, never written into code or committed - **Why:** A leaked key lets anyone make calls billed to you. Keep it in config, never in code, the README, or relying on repo privacy. 3. *What does `npm install groq-sdk` accomplish for the Day-3 rubric?* - **Answer:** It installs the SDK and records it in package.json, which the judge checks for - **Why:** Installing the SDK adds it to package.json dependencies (the judge looks for the import). It doesn't create keys, deploy, or run the model locally. 4. *Where should the Groq call live, and why?* - **Answer:** In a server-side Next.js API route, so the secret key never reaches the browser - **Why:** The call must be server-side: client-side code (and anything bundled into the UI) exposes the key to everyone. Speed and repo privacy don't change that.

Resources. - Groq — quickstart: <https://console.groq.com/docs/quickstart> - Groq — API keys: <https://console.groq.com/keys> - Groq — supported models: <https://console.groq.com/docs/models>

Lesson 3.2 — Prompts & Key Safety

~18 min reading · ends with a required ~2:36 video

Hook. The same model gives a vague answer or a perfect one depending entirely on how you ask. Prompting is the real skill — and keeping your key safe is the real responsibility.

In one line. A good prompt has a clear role, a specific task, and the input data. Read the key with process.env, gitignore .env.local, rotate any key you ever committed, and handle the model's response defensively.

Learning objectives. - **Role + task + input** — Good prompts state who the model is, exactly what to do, and the data to do it with. - **System vs user message** — System = constant persona/rules; user = the changing request and input. - **env var + gitignore + rotate** — process.env.GROQ_API_KEY, gitignore .env.local. Already committed a key? Rotate it now.

Walkthrough (what is taught). 1. **The anatomy of a good prompt** — A prompt is your instruction to the model, and its structure decides the quality of what you get back. 2. **System and**

user messages — The Groq chat API (OpenAI-compatible) takes an array of **messages** with roles. 3. **Keys: process.env, .gitignore, and rotating a leaked key** — Make the key safe, concretely. 4. **Handling the response** — The model returns text, and your app must deal with whatever it gets — not always what you expected.

Required video. A ~2:36 recap plays at the end; the learner must watch it through to complete the lesson and continue.

Teaching notes. - *Emphasize:* Role + task + input. System vs user message. And rotation: gitignoring a leaked key does NOT un-leak it — you must revoke and reissue. - *Common pitfall:* Trusting a careful prompt to guarantee the output format. Treat model output as untrusted. - *Time plan:* ~6 min prompt anatomy, ~6 min system/user messages, ~6 min keys + response handling. - *Discussion prompt:* Draft a system message as a group, then critique one volunteer's.

Knowledge checks (with answers). 1. *What three parts make a reliable prompt?* - **Answer:** A role, a specific task, and the input data - **Why:** Role + task + input turns generic output into useful output. Keys/URLs belong in config, not the prompt; multiple questions confuse the model. 2. *What goes in the system message versus the user message?* - **Answer:** System = constant persona and rules; user = this call's specific request and input - **Why:** The system message locks tone and rules once; the user message carries each call's fresh input. It's not for keys, and the roles aren't interchangeable. 3. *How do you read the key safely, and keep it out of Git?* - **Answer:** Read process.env.GROQ_API_KEY server-side and add .env.local to .gitignore - **Why:** Put the key in .env.local, read it with process.env.GROQ_API_KEY server-side, and gitignore .env.local. Comments, the README, and client code all leak it. 4. *You accidentally committed your API key, then added .env.local to .gitignore. Are you safe?* - **Answer:** No — gitignore doesn't un-leak a committed key; revoke it and generate a new one now - **Why:** A committed secret stays in history; gitignoring it later changes nothing. The only fix is to rotate: revoke the leaked key in the Groq console and issue a new one.

Resources. - Groq — chat completions (text): <https://console.groq.com/docs/text-chat> - Next.js — environment variables: <https://nextjs.org/docs/app/building-your-application/configuring/environment-variables> - Prompt engineering guide: <https://www.promptingguide.ai/>

Day 4 — Build Your AI Feature

Goal. Turn the LLM call into the app's signature feature, running on real user data.

What the learner ships today. The one AI feature, end to end: real input from MongoDB -> prompt -> rendered output.

Day rubric (how it is judged). The AI feature is implemented end to end and does the job its name promises.

By end of day. The student's app does one genuinely useful AI thing on the user's own data.

Lesson 4.1 — Designing Your Signature Feature

~19 min reading · ends with a required ~2:34 video

Hook. This is the day your app earns its name. The AI feature isn't a bolt-on — it's the one thing that makes your project yours.

In one line. Turn your project's archetype and AI feature into a precise job for the model: design the prompt around that job, feed it real user input from MongoDB, and shape the output so your app can use it.

Learning objectives. - **Each archetype, one AI shape** — Feed transforms a post; Tracker reflects on entries; Catalog enriches/matches; Chat converses. - **Design + test the prompt first** — Add quality rules (e.g. "don't invent claims"); iterate in the Groq playground before wiring. - **Real input, shaped output** — Insert live MongoDB data; specify the output format so your code barely cleans up.

Walkthrough (what is taught). 1. **From archetype to feature** — Your project is an archetype plus one AI feature, and today you make that feature real. 2. **Designing the prompt for your job** — With the job defined, design the prompt — this is where Day 3's skill pays off. 3. **Feeding real user input** — A prompt with hard-coded text isn't a feature. 4. **Shaping the output** — The model returns text, but your app needs it in a usable form — so decide the output's shape and make the model produce it.

Required video. A ~2:34 recap plays at the end; the learner must watch it through to complete the lesson and continue.

Teaching notes. - *Emphasize:* Map archetype -> AI shape, write the one-sentence job, and test the prompt in the Groq playground BEFORE wiring any code. - *Common pitfall:* Hard-coded input

(the feature must run on the user's real stored data) and letting the model invent facts — add a guard rule to the prompt. - *Time plan*: ~6 min archetype + job, ~7 min prompt design + playground, ~6 min real input + output shape. - *Discussion prompt*: Each student states their feature: "the model takes ____ and returns ____."

Knowledge checks (with answers). 1. *Where should your AI feature's input come from?* - **Answer**: The data the user already created in your app, read live from MongoDB - **Why**: Using the app's existing data makes the feature feel native, not bolted on. A separate AI-only form or made-up/generic data defeats that. 2. *Why add a rule like "don't add claims they didn't make" to your prompt?* - **Answer**: It stops the model inventing facts, which keeps the feature trustworthy - **Why**: Guarding against invented content prevents a common, trust-killing failure. It's about accuracy, not length, key safety, or speed. 3. *Why test your prompt in the Groq playground before wiring it into code?* - **Answer**: You can iterate on wording fast and confirm it works across inputs before adding code - **Why**: Iterating in the playground first means the wiring is just plumbing. System messages work in code too, prompts stay editable, and both paths use the same free tier. 4. *Why specify the output format explicitly in the prompt?* - **Answer**: So the model returns usable text/JSON and your code barely has to clean it up - **Why**: Precise output shape ("return only the pitch" or strict JSON) makes the feature reliable and the parsing simple. It doesn't change creativity or billing.

Resources. - Groq Playground: <https://console.groq.com/playground> - Groq — chat completions: <https://console.groq.com/docs/text-chat> - Mongoose — queries: <https://mongoosejs.com/docs/queries.html>

Lesson 4.2 — Wiring the Feature End-to-End

~18 min reading · ends with a required ~2:34 video

Hook. A feature is only real when the whole path works: a click in the browser travels to your server, to the model, and back to the screen. Today you connect every link.

In one line. Trace the full request flow — UI to API route to Groq and back — call the model from a Next.js route, handle the Chat archetype's multi-turn messages array, render output with a loading state, and test on several real inputs.

Learning objectives. - **One clean API route** — get input → read MongoDB → build messages → call Groq → return text. The path tells you where to debug. - **Chat = send the messages array** — Multi-turn: send system + alternating user/assistant turns each call; keep only the last ~6–10. - **Loading state + real testing** — Show a "thinking..." state; test short, long, empty, messy inputs and note what's shaky.

Walkthrough (what is taught). 1. **The request flow, and calling Groq from your route** — Your AI feature is a round trip through four places, and seeing the path makes wiring it straightforward. 2. **Multi-turn conversation (the Chat archetype)** — If your archetype is **Assistant/Chat**, your feature isn't one-shot text-in/text-out — it is a conversation, and that changes the wiring. 3. **Rendering output with a loading state** — Make the result visible: render the response where it belongs — the polished pitch beneath the original, the nudge on the dashboard, the latest reply in the chat thread. 4. **Testing on real input** — A feature that works on one input is a coincidence; one that works on several is real.

Required video. A ~2:34 recap plays at the end; the learner must watch it through to complete the lesson and continue.

Teaching notes. - *Emphasize:* The four-link path (UI -> route -> Groq -> UI) is the debugging map. A loading state is non-negotiable. Chat = resend the whole messages array each call. - *Common pitfall:* No loading state (users double-click and assume it broke), and sending the entire chat history every call — keep the last ~6-10 turns. - *Time plan:* ~7 min route + flow, ~5 min the chat case, ~6 min loading state + testing on real inputs. - *Discussion prompt:* Where in your UI does the AI result land?

Knowledge checks (with answers). 1. *What path does an AI feature's request travel?* - **Answer:** UI → API route → Groq → API route → UI - **Why:** The request goes UI → API route → Groq → back. Calling Groq from the browser would expose your key; the route is the required middle. 2. *For a Chat/Assistant feature, how does the model "remember" earlier turns?* - **Answer:** You send the whole messages array (system + prior user/assistant turns) on every call - **Why:** LLMs are stateless, so you resend the full messages array each call. There's no server-side memory or magic flag, and Chat is absolutely doable. 3. *Why show a loading state while the feature runs?* - **Answer:** Model calls take a moment; without feedback users assume it's broken and re-click - **Why:** A "thinking..." state is the difference between "feels broken" and "feels alive." It doesn't affect accuracy and isn't an API requirement. 4. *How do you know your feature is genuinely real, not a coincidence?* - **Answer:** It produces reasonable output across several different inputs without crashing - **Why:** Testing short, long, empty, and messy inputs shows it's real and reveals where guardrails are needed. One success or a clean compile isn't proof.

Resources. - Groq — chat completions API: <https://console.groq.com/docs/text-chat> - Next.js — fetching data: <https://nextjs.org/docs/app/building-your-application/data-fetching> - Mongoose — queries: <https://mongoosejs.com/docs/queries.html>

Day 5 — Polish & Ship

Goal. Make the feature reliable, then ship the whole thing.

What the learner ships today. Guardrails on the AI call, real empty/loading/error states, a working deployed demo, a README, a 30-60s demo clip, and a share post.

Day rubric (how it is judged). Error handling/guardrails on the AI call; a working deployed demo; a clear README; a share post.

By end of day. The student has a shipped, reliable app at a public URL, plus a demo clip and a share post.

Lesson 5.1 — Guardrails & Reliability

~18 min reading · ends with a required ~2:29 video

Hook. Anything that talks to an AI model and the internet will sometimes fail. The difference between a demo and a product is what happens on a bad day.

In one line. Make your AI feature reliable: handle errors and timeouts gracefully, validate what the model returns, respect rate limits and cost, and give every state — loading, empty, error — a real UI.

Learning objectives. - **Make failure boring** — Catch errors and timeouts; show a friendly message, re-enable the button, never crash. - **Treat AI output as untrusted** — Validate fields and format before using; fall back instead of crashing. Prompt well, then verify. - **Three states for every screen** — Give empty, loading, and error each a real UI — that's where finished apps show.

Walkthrough (what is taught). 1. **Handling errors and timeouts** — Your AI call depends on a remote service over the internet, and remote services fail — they go slow, return errors, or occasionally do not answer at all. 2. **Validating the model's output** — An LLM is probabilistic: most of the time it returns what you asked for, occasionally it does not. 3. **Rate limits and cost** — Providers cap how many calls you can make in a window — a **rate limit** — and Groq's free tier is no exception. 4. **Loading, empty, and error states** — A polished app answers a simple question for every screen: what does the user see when there is nothing, when something is happening, and when something went wrong?

Required video. A ~2:29 recap plays at the end; the learner must watch it through to complete the lesson and continue.

Teaching notes. - *Emphasize:* Make failure boring: try/catch + a timeout + a friendly fallback. Validate the output. Give every screen three states: empty, loading, error. - *Common pitfall:* Assuming the call always succeeds. Per-keystroke calls or tight retry loops will trip the rate limit. - *Time plan:* ~6 min errors/timeouts, ~6 min validate output + limits, ~6 min the three states. - *Discussion prompt:* Force a failure (kill the network) and watch exactly what the user sees.

Knowledge checks (with answers). 1. *Why must you handle errors and timeouts around the AI call?* - **Answer:** Remote services fail sometimes; the app should degrade gracefully, not crash - **Why:** Remote calls fail, time out, or return empty — even in production. Catching that and showing a calm message is the difference between a demo and a product. 2. *How should you treat the text an LLM returns?* - **Answer:** As untrusted output — validate the format and any fields before using it - **Why:** A careful prompt helps but never guarantees the shape. Validate fields/format and fall back when it's off — that turns "usually works" into "always behaves." 3. *What's a sensible way to stay within rate limits and run efficiently on a five-day build?* - **Answer:** Trigger the call on a deliberate action and send only the input the model needs - **Why:** Call on a click/submit, send a relevant slice (last 7 check-ins, not all history), and retry at most once or twice. Per-keystroke calls and tight retry loops cause floods. 4. *Which three states separate a finished app from a demo?* - **Answer:** Empty, loading, and error states given a real face on each screen - **Why:** Empty, loading, and error states are where finished apps visibly differ. Themes and breakpoints are separate concerns.

Resources. - Groq — rate limits: <https://console.groq.com/docs/rate-limits> - MDN — try...catch: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/try...catch> - Next.js — error handling: <https://nextjs.org/docs/app/building-your-application/routing/error-handling>

Lesson 5.2 — Ship & Share

~17 min reading · ends with a required ~2:52 video

Hook. The last mile is the one most people skip — and it's the one everyone sees. A live URL, a clear README, and a sharp demo are what turn five days of work into something real.

In one line. Ship the final build: confirm the deployed demo works for a stranger, write a README that explains and runs the project, record a short demo, and share it. Shipping is a skill — finish strong.

Learning objectives. - **The live URL is the deliverable** — Test the deployed app as a fresh stranger: open it, create, run the AI feature. Live beats local. - **README is the front door** — Title,

pitch, what-it-is, demo link, how-to-run (env vars named), how-the-AI-works. The judge reads it. - **Demo + share post** — A 30–60s clip of the critical path, then a proud, simple post with the link. Shipping is the skill.

Walkthrough (what is taught). 1. **The deployed demo** — The single most important artefact on Day 5 is a working public URL. 2. **Writing your README** — Your README is the front door to your repo, and on Day 5 the AI judge reads it to understand what you built. 3. **Recording a demo** — A short demo video or set of screenshots captures your app at its best and travels further than a URL alone. 4. **The share post** — The final step closes the loop: a public post that tells the world what you built.

Required video. A ~2:52 recap plays at the end; the learner must watch it through to complete the lesson and continue.

Teaching notes. - *Emphasize:* The live URL is the deliverable. The README is the front door (env vars named, never valued). The demo clip is 30-60s of the critical path. - *Common pitfall:* A beautiful local build that fails at its public URL does not exist for judging. Never put real keys in the README. - *Time plan:* ~6 min deploy check, ~6 min README, ~5 min demo clip + share post. - *Discussion prompt:* Watch each other's 60-second demos and give one piece of feedback each.

Knowledge checks (with answers). 1. *How should you test your deployed demo on Day 5?* - **Answer:** Open the public Vercel URL in a fresh/incognito browser and run the full critical path end to end - **Why:** Only the live URL, exercised as a stranger, counts. A build that runs locally but fails at its public URL is, for judging, a build that doesn't exist — and the Day-2 deploy can drift. 2. *What makes a strong project README the judge can use?* - **Answer:** Pitch, what-it-is, a prominent live demo link, how-to-run with env vars named, and how the AI feature works - **Why:** A clear README (pitch, demo link, run steps with env vars named but not valued, AI explanation) addresses the rubric. Real keys/URIs must never be committed; a file listing helps no one. 3. *What should a 30–60 second demo clip show?* - **Answer:** The critical path — open, core action, AI result — with the AI moment as the highlight - **Why:** A tight clip of open → core action → AI result makes your project's value obvious in under a minute. Code scrolls and unfocused recordings bury the point. 4. *What belongs in your final share post?* - **Answer:** One line on what you built, the AI feature, what you learned, and a link to the live demo - **Why:** A simple, proud post — what you built, the AI feature, a learning, and the live link — closes the loop. No code dumps, no apologising, never secrets.

Resources. - Deploy & redeploy on Vercel: <https://vercel.com/docs/deployments/overview> - How to write a good README: <https://www.makeareadme.com/> - Groq — docs home: <https://console.groq.com/docs>

Glossary

- **Next.js** — A React framework. One project holds both the UI (pages under app/) and the backend (API routes under app/api/.../route.ts).
- **App Router** — Next.js's file-based routing where folders under app/ become pages and routes.
- **API route** — A server-side file (route.ts) that handles a request (GET/POST/etc.) — where database and AI calls live, away from the browser.
- **Node.js** — The runtime that runs your Next.js app on your machine.
- **Git / GitHub** — Version control (Git) and the hosted repo (GitHub) the AI judge reads.
- **Vercel** — Hosting that connects your GitHub repo and auto-deploys it to a public URL on every push.
- **MongoDB Atlas** — A cloud-hosted MongoDB database (free M0 tier) where the app stores its data.
- **Mongoose** — A library for talking to MongoDB: you define a schema once and get a model with methods to create and query documents.
- **Schema / Model** — A schema is the named list of fields for your data; the model is the object you call (find/create/etc.).
- **Document / Collection** — MongoDB stores JSON-like records (documents) inside collections (the equivalent of tables).
- **CRUD** — Create, Read, Update, Delete — the four operations on your data.
- **Environment variable / .env.local** — A secret value (like a key or DB URI) read at runtime, kept out of code and out of Git.
- **MONGODB_URI** — The secret connection string for MongoDB Atlas (contains the DB password). Server-side only.
- **Groq** — The AI provider used here; runs open models (e.g. Llama) with an OpenAI-compatible API and a generous free tier.
- **groq-sdk / GROQ_API_KEY** — The Groq client library and the secret key that authorizes calls.
- **LLM** — Large language model — a service you send a prompt to and get generated text back.
- **Prompt / Completion** — The prompt is your instruction + input; the completion is the model's generated reply.
- **System vs user message** — System sets the constant persona/rules; the user message carries each call's changing input.
- **Archetype** — The shape of the app: Board/Feed, Tracker/Log, Catalog/Directory, or Assistant/Chat — each maps to a natural AI feature.

- **Gating video** — A required end-of-lesson video; the lesson cannot complete until it is watched through.
- **Day-gate** — Access control that unlocks a day's lessons on that event day; future days stay locked.

Stack Cheat-Sheet

```
Accounts (free):  GitHub · MongoDB Atlas · Vercel · Groq
Install locally:  Git · Node.js (LTS) · an AI coding assistant
Secrets (.env.local, never committed):
  MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/app
  GROQ_API_KEY=gsk_...
First LLM call:   client.chat.completions.create({ model: "llama-3.3-70b-
versatile", messages: [...] })
Deploy walls:     (1) set env vars in Vercel  (2) Atlas Network Access
0.0.0.0/0  (3) exact connection string
```